

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA PRÁCTICA – Práctica 1

Presentación

En la PEC 1 se han analizado diferentes estilos arquitectónicos y los criterios para seleccionar cuál o cuáles de ellos son los más adecuados para guiar el desarrollo de una solución.

En esta Práctica 1 veremos cómo diseñar una solución basada en una arquitectura de microservicios, usando modelos de representación y patrones de diseño.

La Práctica 1 abarca los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En la Práctica 1 se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Capacidad de analizar un problema en el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y resolverlo.

Objetivos

Los objetivos de la Práctica 1 son:

- Identificar operaciones que un sistema tiene que proporcionar.
- Proponer el diseño de una arquitectura y su descomposición en microservicios.
- Diseñar de forma detallada un microservicio.
- Evaluar y justificar cómo resolver una problemática concreta en el diseño de microservicios (escoger un patrón, un estilo de comunicación o resolver la transaccionalidad).

Descripción de la Práctica

Tras una nueva ronda de reuniones con el grupo inversor, están muy satisfechos con el estilo arquitectónico basado en microservicios que hemos propuesto para **Photo&Film4You** en la primera actividad del curso (PEC1). Sin embargo, se nos ha pedido añadir nuevos requisitos y algunas aclaraciones/restricciones al catálogo de alquiler de material audiovisual:

- Para el alquiler de materiales se utilizarán los estados "planificados", "cancelados" y "devueltos". Los escenarios en los que el cliente no devuelve el material alquilado o lo devuelve en condiciones defectuosas permanecen fuera del alcance del proyecto.
- Las categorías y subcategorías no pueden ser modificadas ni eliminadas.
- Las descripciones de productos siempre pueden actualizarse; sin embargo, el precio del alquiler diario solo puede modificarse si no hay unidades de producto comprometidas para alquilar.
- Un producto solo puede ser dado de baja de nuestro sistema si no está comprometido actualmente para ningún alquiler. Esta acción desregistrará tanto el producto como todas las unidades asociadas. Desde la perspectiva del usuario, estos productos no están disponibles, es como si estuvieran eliminados. Sin embargo, no se eliminan de nuestra base de datos. En su lugar, se les marca el estado de "no activo".
- De manera similar, las unidades producto pueden designarse como "no operativas" exclusivamente cuando no están involucradas en ningún contrato de alquiler.
- En el ámbito tecnológico, nos piden que la persistencia de datos de la plataforma debe gestionarse usando JPA y PostgreSQL.

Por otro lado, se solicita a la empresa incorporar dos nuevos servicios para mejorar la experiencia del usuario. Estos están descritos en el enunciado general del proyecto de este curso:

- Gestión de cursos de formación audiovisual
- Gestión de microcredenciales académicas respecto los cursos completados por un alumno.

Respecto el primer servicio, consiste en un módulo para gestionar cursos de formación audiovisual, en el que los usuarios puedan mejorar sus habilidades y aprovechar mejor el equipo disponible. Para simplificar este caso de uso:

- Se utilizará un Sistema de Gestión de Aprendizaje (LMS) externo de código abierto, que se personalizará para proporcionar gestión de cursos y ofrecer contenidos, foros, actividades y otros recursos de aprendizaje.
- No se debe realizar ningún trabajo adicional de desarrollo dentro de este LMS para este enunciado, pues nuestro enfoque principal está en el propio sistema de gestión académica. Concretamente: inscripciones de cursos, registros de cursos, perfiles de los miembros y una hoja detallada que indique el título, descripción, modo de impartición (presencial, online o híbrido), precio, idioma, ubicación, objetivos, metodología y otra información relevante.

- Los usuarios pueden explorar los cursos disponibles, consultar los detalles (metodología, objetivos...) y matricularse en un curso específico.
- El profesor responsable puede consultar a los estudiantes matriculados y modificar los detalles del curso, incluyendo el título, descripción, modalidad, objetivos, metodología, duración y lenguaje. Además, el profesor es responsable de cerrar los informes de calificaciones al final del curso
- Los administradores pueden crear cursos (asociados a un profesor existente), abrir y cerrar periodos de inscripción, y cerrar registros de cursos, entre otras acciones.

Por otro lado, respecto el servicio de generación de microcredenciales, la petición debe producirse en el momento que un alumno complete un curso y reciba la calificación. El sistema debe ser capaz de mostrar al administrador todas las peticiones pendientes de tratar, y procesar su aprobación de manera individual o por lotes. Una vez confirmada una microcredencial, el alumno es notificado y la credencial quedaría disponible en el expediente del alumno.

- Por simplificar: la generación de la microcredencial e incorporación en el expediente, será realizada por un sistema externo, quedando fuera del alcance de este proyecto.
- La petición de generación de la credencial se realizará de manera síncrona. La notificación al administrador para confirmar la credencial, y al alumno para indicar que ya está tramitada en su expediente, si que debe ser asíncrona.

En resumen, tal como explica el enunciado, el flujo de trabajo académico consta de las siguientes etapas: apertura de matrículas (administrador), cierre de matrículas (administrador), impartición de cursos (usuarios de instructores y estudiantes), cierre de informes de calificación (instructor), cierre de cursos, generación de microcredenciales y aceptación de éstas (administrador).

Ejercicio 1 (2.5 puntos)

Basándose en todos los requisitos establecidos en la **especificación Photo&Film4You**, especificar la arquitectura de microservicios de la aplicación indicando:

1. Operaciones del sistema (1 punto). Usando el patrón CQRS, separa los **comandos** de las **consultas**:

Para cada uno de los **comandos** especifica:

- Actor
- Caso de uso
- Nombre del mando
- Descripción

Para cada uno de los **Consultas** especifica:

- Actor
- Caso de uso
- Nombre de la consulta
- Descripción

NOTA: Debe usarse una nomenclatura "Lower Camel Case"

(<https://wiki.c2.com/?LowerCamelCase>) para los nombres de comandos y consultas.

2. Especificación de comandos (1 punto). Para cada comando, especifica sus parámetros, su valor de retorno, sus precondiciones y sus postcondiciones.

3. Modelo general de dominio (1 punto). Diagrama de alto nivel del modelo de dominio.

NOTA: Puedes encontrar un ejemplo del diagrama solicitado en el libro "Microservices Patterns", capítulo 2: "Estrategias de descomposición", figura 2.7: "Las clases clave en el modelo de dominio FTGO".

Solución

1. Operaciones del sistema

Los clientes web se comunicarán vía HTTP, pero puede haber clientes que se comuniquen mediante mensajería (colas de mensajes). Para evitar estar atados a un protocolo específico, primero creamos esta abstracción en las operaciones del sistema para representar las diferentes solicitudes que nuestro sistema puede recibir.

Dividimos las operaciones del sistema, usando el patrón CQRS, en:

- Órdenes. Operaciones del sistema que crean, actualizan o eliminan datos.
- Consultas. Operaciones del sistema que leen o consultan datos.

Comandos:

Actor	Caso de uso	Comando	Descripción
Administrador	Añadir una sección o subsección	createCategory()	Crea una sección o subsección en el catálogo
Administrador	Añadir un producto	createProduct()	Crea un producto dentro del catálogo
Administrador	Modificar una descripción de	updateProductDescription()	Modificar la descripción de un producto dentro del

	producto		catálogo
Administrador	Modifica el precio diario del alquiler de un producto	updateProductDailyPrice()	Modifica el precio diario del alquiler de un producto dentro del catálogo
Administrador	Eliminar un producto	deleteProduct()	Eliminar un producto del catálogo
Administrador	Añade una unidad de un producto	createItem()	Crea una unidad de un producto dentro del catálogo
Administrador	Configura una unidad de producto como operativa	setOperational()	Configura una unidad de producto como operativa y disponible para alquilar de nuevo
Administrador	Añade un curso	createCourse()	Crea un curso
Administrador	Abrir el periodo de inscripción	openEnrollment()	Abre el periodo de matrícula para un curso específico
Administrador	Cerrar el periodo de inscripción	closeEnrollment()	Cierra el periodo de matrícula para un curso específico
Administrador	Cierra un curso	closeCourse()	Cierra un curso
Administrador	Aprobar petición de microcredencial	approvePendingMicrocredential()	Marca una petición de microcredencial pendiente como correcta, y se incorporarse al expediente
Administrador	Rechazar petición de microcredencial	rejectPendingMicrocredential()	Marca una petición como incorrecta
Usuario no registrado	Registro en el sistema	signUp()	Registro en el sistema
Usuario	Modificar información personal	updateUserProfile()	Actualizar información personal
Usuario	Crear un usuario	createUser()	Crear un usuario

Usuario	Crea un carrito nuevo	createCart()	Crea un nuevo carrito con un rango de fechas y una unidad de producto especificados
Usuario	Añade una unidad al carrito	addItemToCart()	Añadir una unidad de producto al carrito para los rangos de fechas especificados
Usuario	Saca una unidad del carro	deleteItemFromCart()	Retira una unidad de producto del carrito
Usuario	Envía una solicitud de alquiler	checkoutRent()	Crea una solicitud de alquiler con el contenido del carrito
Usuario	Cancelar una solicitud de alquiler	cancelRent()	Cancelar una solicitud de alquiler
Usuario	Crea una alerta	createAlert()	Crea una alerta sobre la disponibilidad de un producto en un rango de fechas
Usuario	Eliminar una alerta	dropAlert()	Elimina una alerta sobre la disponibilidad de un producto en un rango de fechas
Usuario	Inscribirse en un curso	enrollInCourse()	Inscribe a un usuario específico en un curso determinado.
Usuario (instructor)	Modificar los detalles del curso	modifyCourseDetails()	Actualiza los detalles de un curso específico.
Usuario (instructor)	Cerrar informe de calificaciones	closeGradeReports()	Cierra los informes de calificaciones de un curso específico.
Funcionamiento del sistema, no disponible para los usuarios	solicitar microcredenciales para un curso completado	requestCourseMicrocredentials()	Envía una petición de generación de microcredenciales, para un curso ya completado con sus evaluaciones

Funcionamiento del sistema, no disponible para los usuarios	Enviar notificación	<code>notifyProductAvailable()</code> <code>notifyCredentialPending()</code> <code>notifyCredentialGranted()</code>	Enviar notificación por una disponibilidad de producto, una microcredencial pendiente de revisión, o una confirmada e incorporada al expediente

Consultas:

Actor	Caso de uso	Consulta	Descripción
Usuario no registrado	Ver categorías/ subcategorías disponibles	<code>findCategoriesByCriteria()</code>	Consultar las categorías y subcategorías disponibles en el catálogo por nombre, descripción o categoría principal
Usuario no registrado	Buscar productos por criterios	<code>findProductsByCriteria()</code>	Busca productos por varios criterios (categoría, nombre, descripción, marca, modelo)
Usuario no registrado	Ver información del producto	<code>getProductById()</code>	Obtener información sobre un producto, incluidas sus unidades
Usuario no registrado	Ver información sobre una unidad	<code>getItemById()</code>	Obtén información sobre una unidad de producto, incluyendo solicitudes de alquiler activas
Usuario no registrado	Ver unidades disponibles en un rango de fechas	<code>findRentAvailableItems()</code>	Consulta las unidades de un producto disponibles para alquilar en un rango de fechas especificado

Usuario no registrado	Encontrar cursos	findCourses()	Búsquedas de cursos
Usuario no registrado	Ver información del curso	getCourseById()	Obtén información sobre un curso
Usuario	Ver información personal	getUserProfile()	Recuperar información personal
Usuario	Ver información del carrito	getCart()	Consulta el contenido del carro
Usuario	Consulta las alertas de disponibilidad del usuario	findAlertsByUser()	Consulta las alertas de disponibilidad de un producto en un rango de fechas para un usuario
Usuario	Consulta la información sobre una alerta de disponibilidad	getAlertById()	Consulta la información sobre una alerta de disponibilidad
Funcionamiento del sistema, no disponible para los usuarios	Recibe las alertas por usuario	getAlertsByUserAndInterval()	Recibe las alertas por usuario
Funcionamiento del sistema, no disponible para los usuarios	Recibe las alertas por producto	getAlertsByProductAndDate()	Recibe las alertas por producto
Usuario	Consulta las solicitudes de alquiler del usuario	findRentsByUser()	Consulta las solicitudes de alquiler de un usuario
Usuario (instructor)	Recupera la lista de estudiantes matriculados	getEnrolledStudents()	Recupera la lista de estudiantes matriculados en un curso específico

Usuario (instructor)	Recupera la lista de matriculaciones por curso	getEnrollmentByCourse()	Recupera la lista de matriculaciones por curso
Usuario / Administrador	Consulta la información en una solicitud de alquiler	getRent()	Consulta la información sobre una solicitud de alquiler
Administrador	Ver las microcredenciales pendientes	getPendingMicrocredentialRequests()	Consulta las microcredenciales pendientes de aprobar o rechazar

*Nota: Ninguna operación del sistema relacionada con el inicio de sesión se representa, ya que la autenticación la gestiona el **componente API Gateway**, por lo que no se considera un requisito real del dominio. Las soluciones que han representado operaciones de inicio de sesión se consideran correctas.*

2. Especificación de comandos

- createCategory

Operación: createCategory(name, description, parentCategoryId)

Retorno: categoryId

Condiciones previas:

- No existe ninguna categoría ya existente con el mismo nombre.
- Si se especifica una categoría madre, debe existir.

Condiciones posteriores:

- Se crea la categoría.
- Si se especifica una categoría madre, se establece una relación madre-hija entre ambas.

- createProduct

Operación: createProduct(categoryId, name, description, brand, model, dailyPrice)

Retorno: productId

Condiciones previas:

- La categoría ya existe.
- No hay ningún producto ya existente con el mismo nombre.

Condiciones posteriores:

- El producto se crea sin ninguna unidad como producto dentro de la categoría especificada.

- **updateProductDescription**

Operación: updateProductDescription (productId, newDescription)

Retorno: Verdadero/Falso

Condiciones previas:

- El producto ya existe.

Condiciones posteriores:

- La descripción del producto se actualiza al valor especificado.

- **updateProductDailyPrice**

Operación: updateProductDailyPrice (productId, newDailyPrice)

Retorno: Verdadero/Falso

Condiciones previas:

- El producto ya existe.
- El producto no tiene unidades comprometidas en alquileres activos.

Condiciones posteriores:

- El precio diario del producto se actualiza al valor especificado.

- **deleteProduct**

Operación: deleteProduct(productId)

Retorno: Verdadero/Falso

Condiciones previas:

- El producto ya existe.
- El producto no tiene unidades comprometidas en alquiler.

Condiciones posteriores:

- El producto y todas sus unidades asociadas se actualizan a estado INACTIVO.

- **createItem**

Operación: createItem(productId, serialNumber)

Retorno: itemId

Condiciones previas:

- El producto ya existe.
- No hay ninguna unidad ya existente con ese número de serie.

Condiciones posteriores:

- Se crea una unidad producto en estado OPERATIVO sin reservas de alquiler.
- Se envía una notificación por correo electrónico de la nueva unidad a todos los usuarios que tengan una alerta de disponibilidad para ese producto, independientemente de la fecha de la alerta.

- **setNotOperationalItem**

Operación: setNotOperationalItem(itemId)

Retorno: Verdadero/Falso

Condiciones previas:

- La unidad de producto ya existe y está en estado OPERATIVO.
- La unidad del producto no está comprometida con ningún alquiler.

Condiciones posteriores:

- La unidad producto está configurada en estado NO_OPERATIVO y no puede alquilarse.

- **setOperationalItem**

Operación: setOperationalItem(itemId)

Retorno: Verdadero/Falso

Condiciones previas:

- La unidad de producto ya existe y está en estado NO_OPERATIVO.

Condiciones posteriores:

- La unidad de producto está configurada en estado OPERATIVO y puede alquilarse.

- **createCourse**

Operación: createCourse(title, description, enrollmentStartDate, enrollmentEndDate, mode, price, objectives, methodology, duration, language, location, instructorId)

Retorno: courseId

Condiciones previas:

- No hay ningún curso ya existente con el mismo título. El instructor es un usuario que ya existe en el sistema.

Condiciones posteriores:

- El curso está creado. El número de cursos del instructor se ha incrementado en uno.
- El estado del curso está establecido como 'draft', lo que significa que el periodo de inscripción aún no ha comenzado.

- **openEnrollment**

Operación: `openEnrollment(courseId, inicioFecha, finFecha)`

Retorno: vacío

Condiciones previas:

- El curso ya existe.

Condiciones posteriores:

- Se crea una inscripción. Si ya hay un periodo de inscripción abierto, se reportará un error.
- El estado del curso cambia a 'matrícula abierta'.

- **closeEnrollment**

Operación: `closeEnrollment(courseId)`

Retorno: vacío

Condiciones previas:

- El curso ya existe.

Condiciones posteriores:

- La matrícula está cerrada. Si no hay un periodo de inscripción abierto en ese momento, se reportará un error.
- El estado del campo sigue siendo 'activo'.

- **closeCourse**

Operación: `closeCourse(courseId)`

Retorno: Verdadero/Falso

Condiciones previas:

- El curso ya existe.

Condiciones posteriores:

- El curso solo se cierra si todos los registros de los estudiantes han sido procesados con éxito; de lo contrario, no estará completamente cerrado.

- El estado del curso (Course) cambia a 'CLOSED', y también el de sus respectivas inscripciones (Enrollments).

- **requestCourseMicrocredentials**

Operación: requestCourseMicrocredential(courseId)

Retorno: Verdadero/Falso

Condiciones previas:

- El curso existe, las evaluaciones han sido realizadas por el instructor, y el curso ha sido cerrado.

Condiciones posteriores:

- Se crea una petición de microcredenciales para cada enrollment del curso. Estas peticiones quedan en estado REQUESTED, pendientes de ser aprobadas. Se lanza una notificación para los administradores.

- **approvePendingMicrocredential**

Operación: approvePendingMicrocredential(microcredentialId)

Retorno: Verdadero/Falso

Condiciones previas:

- Microcredencial existente y en estado REQUESTED

Condiciones posteriores:

- El administrador aprueba la microcredencial (generada por un sistema externo) y queda en estado GRANTED, se actualiza la fecha y se notifica al alumno al respecto.

- **rejectPendingMicrocredential**

Operación: rejectPendingMicrocredential(microcredentialId)

Retorno: Verdadero/Falso

Condiciones previas:

- Microcredencial existente y en estado REQUESTED

Condiciones posteriores:

- El administrador rechaza la microcredencial (generada por un sistema externo) y queda en estado REJECTED.

- **signUp**

Operación: signUp(email, password, fullName, phoneNumber)

Retorno: Verdadero/Falso

Condiciones previas:

- Ningún usuario con ese correo existe en el sistema.

Condiciones posteriores:

- El usuario está registrado en el sistema y está operativo.

- **updateUserProfile**

Operación: updateUserProfile(email, password, fullName, phoneNumber)

Retorno: Verdadero/Falso

Condiciones previas:

- El usuario identificado por el correo electrónico dado ya existe.

Condiciones posteriores:

- El nombre completo y el número de teléfono del perfil de usuario identificado por el correo electrónico se actualizan con los valores indicados.

- **createCart**

Operación: createCart(email, itemId, from, to)

Retorno: cartId

Condiciones previas:

- El usuario identificado por el correo electrónico dado ya existe.
- La unidad del producto existe y está disponible para alquilar en las fechas indicadas.

Condiciones posteriores:

- Si el usuario no tiene carrito, se crea uno nuevo con las fechas de inicio y finalización especificadas del alquiler.
- Si el usuario tenía un carrito con fechas diferentes, el anterior se elimina y se crea uno nuevo con las fechas de inicio y finalización especificadas.
- La unidad de producto se añade al carrito del usuario.
- La cantidad total se calcula multiplicando el precio diario del alquiler del producto al que pertenece la unidad por el número de días de alquiler.

- **addItemToCart**

Operación: addItemToCart(email, articleId, cartId)

Retorno: cartId

Condiciones previas:

- El usuario identificado por el correo electrónico dado ya existe.
- El carro existe.
- La unidad del producto existe y está disponible para alquilar en las fechas especificadas.

Condiciones posteriores:

- La unidad de producto se añade al carrito del usuario.
- El importe total se actualiza sumando el precio diario del alquiler del producto al que pertenece la unidad multiplicado por el número de días de alquiler.

- **deleteItemFromCart**

Operación: deleteItemFromCart (email, articleId, cartId)

Retorno: Verdadero/Falso

Condiciones previas:

- El usuario identificado por el correo electrónico dado ya existe.
- El carro existe.
- La unidad del producto existe y está en el carrito del usuario.

Condiciones posteriores:

- La unidad de producto se retira del carrito del usuario.
- La cantidad total se actualiza restando el precio diario de alquiler del producto al que pertenece la unidad multiplicado por el número de días de alquiler.

- **checkoutRent**

Operación: checkoutRent(cartId)

Retorno: orderId

Condiciones previas:

- El carro existe.

Condiciones posteriores:

- Se crea una solicitud de alquiler para las fechas y unidades de producto que aparecen en el carrito del usuario.
- El importe total del carrito está autorizado por el sistema de pago externo.
- La solicitud de alquiler está en estado PROGRAMADO y las unidades de producto del carrito tienen una reserva de alquiler para las fechas especificadas.

- **cancelRent**

Operación: cancelRent(rentId)

Retorno: Verdadero/Falso

Condiciones previas:

- La solicitud de alquiler existe.

Condiciones posteriores:

- La solicitud de alquiler está en estado CANCELADO y las unidades producto de la solicitud de alquiler están disponibles para las fechas de alquiler canceladas.
- Se envía una notificación por correo electrónico informando que las unidades están disponibles a todos los usuarios que tengan una alerta de disponibilidad para ese producto en las fechas que tenía la solicitud de alquiler.

- **createAlert**

Operación: createAlert(email, productId, from, to)

Retorno: alertId

Condiciones previas:

- El producto ya existe.
- El usuario identificado por el correo electrónico dado ya existe.

Condiciones posteriores:

- Se crea una nueva alerta asociada al usuario identificado por correo electrónico que notificará cuando las unidades del producto identificadas por el Identificador del producto estén disponibles para las fechas indicadas.

- **dropAlert**

Operación: dropAlert(email, alertId)

Retorno: Verdadero/Falso

Condiciones previas:

- La alerta identificada por alertId ya existe y está asociada con el usuario identificado por el correo electrónico indicado.

Condiciones posteriores:

- La alerta identificada por alertId se elimina del sistema, y el usuario identificado por correo electrónico ya no será notificado cuando las unidades estén disponibles para el producto en las fechas especificadas por la alerta.

- **enrollInCourse**

Operación: enrollInCourse(courseId, email)

Retorno: enrollmentId

Condiciones previas:

- El usuario identificado por el correo electrónico dado existe.
- El curso identificado por el curso existe.

Condiciones posteriores:

- El curso cuenta con un nuevo estudiante matriculado.

- **modifyCourseDetails**

Operación: modifyCourseDetails(courseId, title, description, mode, objectives, methodology, duration, language)

Retorno: Verdadero/Falso

Condiciones previas:

- El curso identificado por courseId existe.

Condiciones posteriores:

- Detalles del curso actualizados.

- **closeGradeReports**

Operación: closeGradeReports(courseId)

Retorno: Verdadero/Falso

Condiciones previas:

- El curso identificado por el curso existe.

Condiciones posteriores:

- El estado del campo cambia a 'pendiente de cierre'.

- **notifyCredentialPending**

Operación: notifyCredentialPending(microcredentialId)

Retorno: Verdadero/Falso

Condiciones previas:

- Microcredencial existente y en estado REQUESTED

Condiciones posteriores:

- El servicio notifica a los administradores del sistema que hay una nueva microcredencial pendiente de aprobar.

- **notifyCredentialGranted**

Operación: notifyCredentialGranted(microcredentialId)

Retorno: Verdadero/Falso

Condiciones previas:

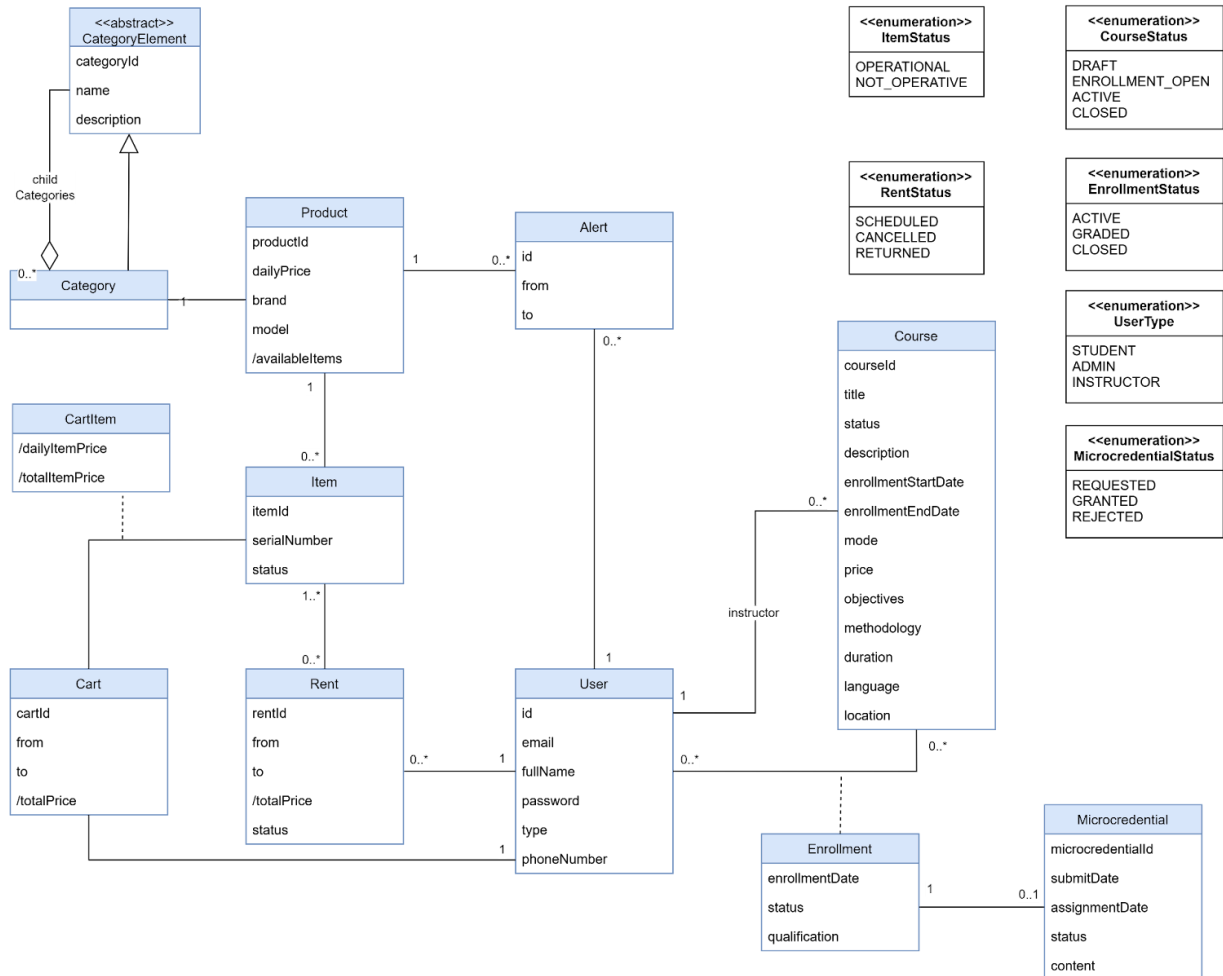
- Microcredencial existente y en estado GRANTED

Condiciones posteriores:

- El servicio notifica al alumno relacionado con la matrícula que ha obtenido la microcredencial, que ésta ha sido aceptada e incorporada a su expediente.

3. Modelo de dominio (de alto nivel)

El modelo de dominio de alto nivel describe las clases clave utilizando el vocabulario con el que describimos las operaciones del sistema.



Notas:

- Modelar marcas y modelos de productos como entidades se considerará correcto.
- Modelar las categorías sin el patrón de diseño "Composite" también se considera correcto.

Referencias:

- Libro "Microservices Patterns": Sección 2.2.1 "Creación de un modelo de dominio de alto nivel".
- Libro "Microservices Patterns": Sección 2.2.1 "Definición de las operaciones del sistema".

Ejercicio 2 (1.5 puntos)

Basándose en la definición de las operaciones del sistema y el modelo de dominio del ejercicio anterior, realizar la descomposición de los diferentes **microservicios** de la aplicación.:

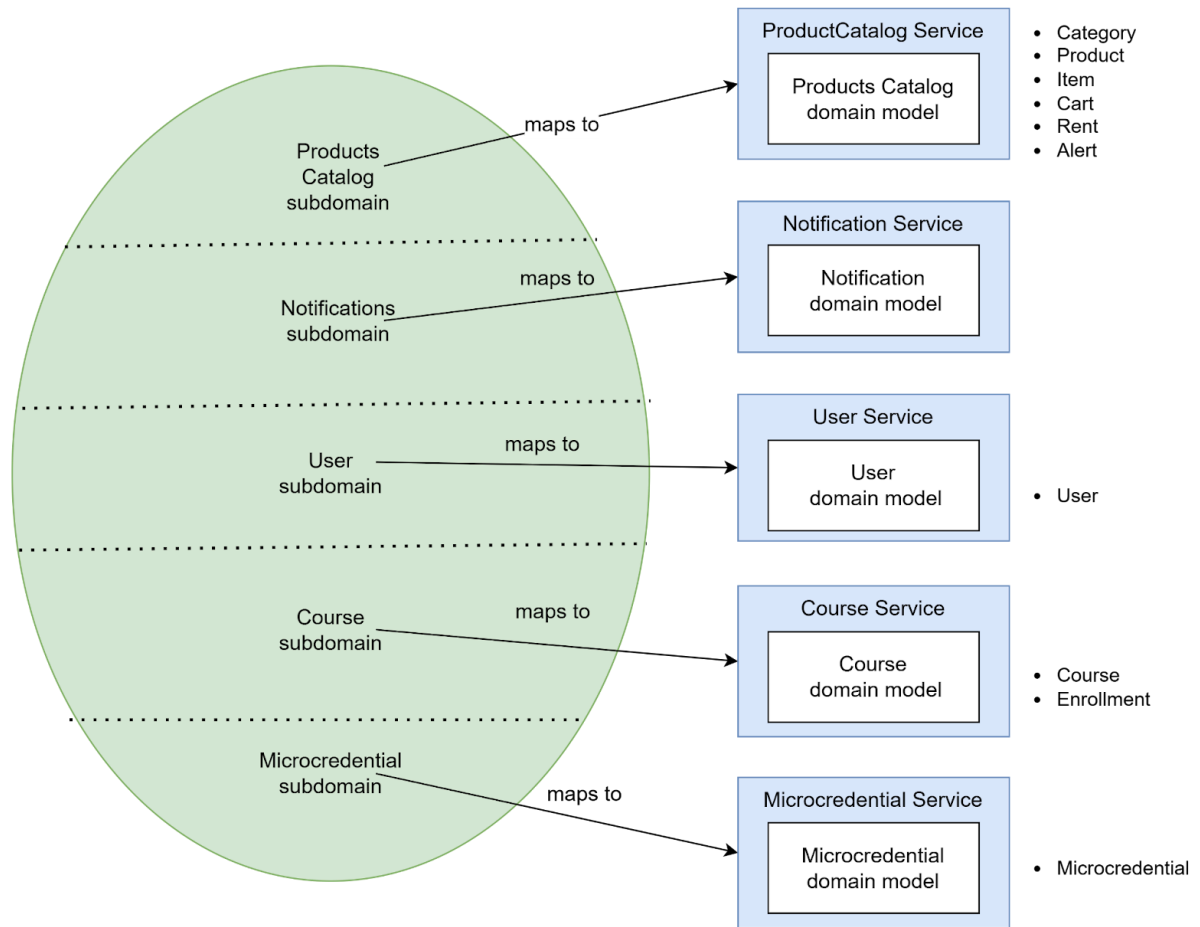
1. Identificar los subdominios y sus **servicios** necesarios, utilizando una descomposición basada en el patrón de subdominios (0,75 puntos).
2. Realizar el **mapeo** entre las operaciones del sistema y los servicios identificados, e identificar las colaboraciones que requieren algunos de los servicios restantes (0,75 puntos).

Solución

1. A continuación, se presenta una posible descomposición del sistema en subdominios y los **servicios** necesarios, utilizando una descomposición basada en el patrón de subdominio descrito en el Capítulo 2, Sección 2.2.3 *"Definiendo servicios aplicando el patrón Descomponer por subdominio"* del libro de referencia.

Ten en cuenta que esta es una posibilidad entre otras, como se discute en el Capítulo 1, Sección 1.5.2 *"Desventajas de la arquitectura de microservicios"* del libro de referencia:

"Uno de los retos de usar la arquitectura de microservicios es que no existe un algoritmo concreto y bien definido para descomponer un sistema en servicios. Como ocurre con gran parte del desarrollo de software, es algo así como un arte"



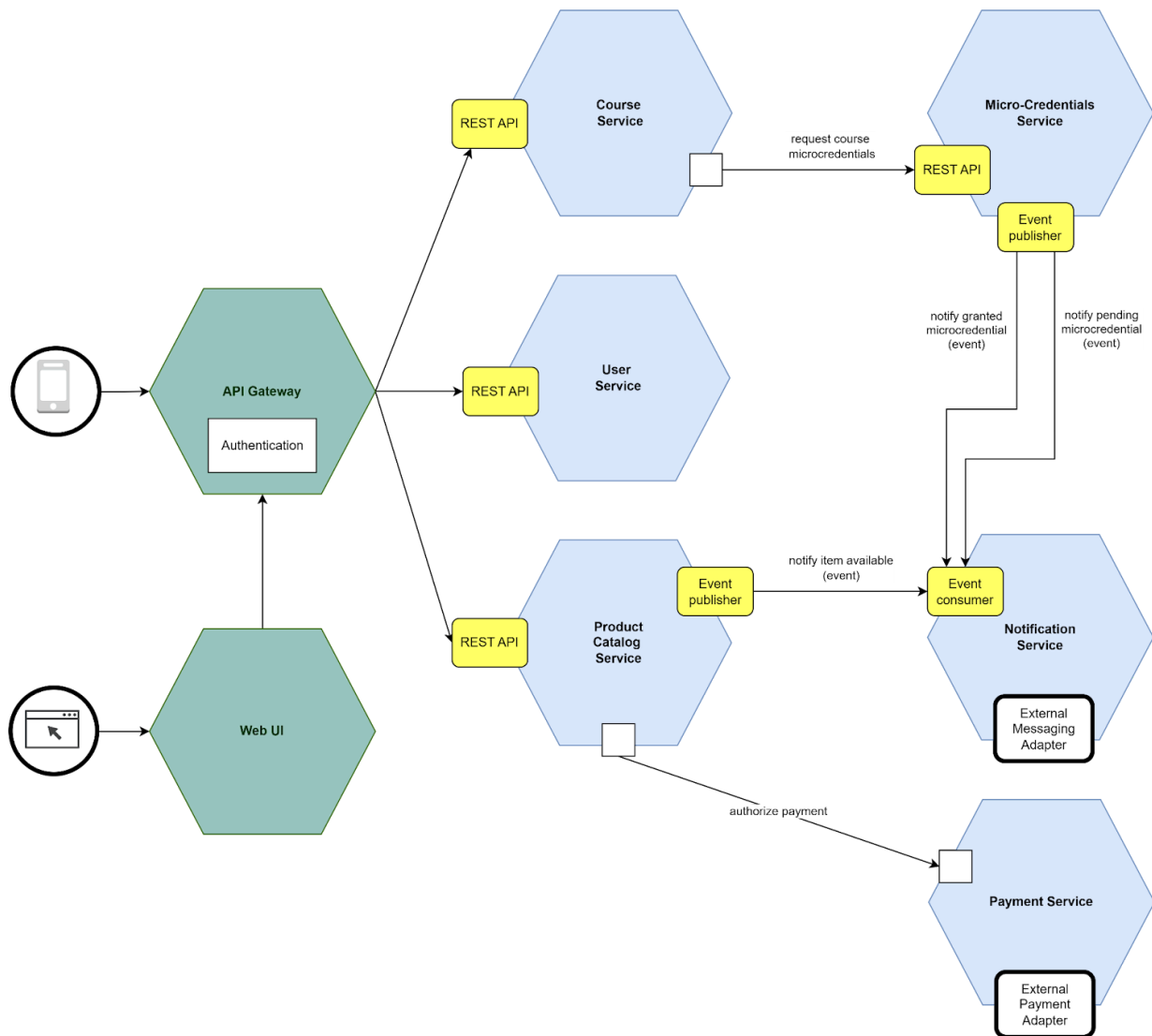
- **Servicio de catálogo de productos:** gestiona el catálogo de equipos y alquileres
- **Servicio de notificaciones:** servicio cruzado de notificaciones a usuarios:
 - o alertas activas sobre la disponibilidad de un artículo del catálogo
 - o petición de microcredenciales pendientes de aprobar
 - o microcredencial de alumno correctamente incorporada
- **Servicio de usuarios:** gestionar el registro de usuarios
- **Servicio de cursos:** gestiona los cursos a lo largo de todo el flujo de trabajo, desde la creación hasta el cierre, incluyendo las matriculaciones de los alumnos
- **Servicio de microcredenciales:** gestiona la generación de credenciales para cursos completados y calificados por alumnos

2. Para cada servicio identificado, determina las operaciones del sistema que resuelve, e identifica las colaboraciones que requieren otros servicios.

Servicio	Operaciones	Colaboradores
Servicio ProductCatalog	- createCategory() - createProduct() - updateProductDescription() - updateProductDailyPrice() - deleteProduct () - createItem() - setNotOperationalItem() - setOperationalItem() - findCategoriesByCriteria() - findProductsByCriteria() - getProductDetails() - getItemDetails() - findRentAvailableItems() - addCart() - addItemToCart() - deleteItemFromCart() - getCart() - checkoutRent() - cancelarRent() - getRent() - findRentsByUser()	Servicio de pago: authorizePayment()

	▪ <code>getRentReservations()</code> ▪ <code>hasActiveReservations()</code>	
Servicio Notifications	▪ <code>notifyCredentialPending()</code> ▪ <code>notifyCredentialGranted()</code> ▪ <code>notifyProductAvailable</code>	Servicio de notificaciones externo: ▪ <code>sendNotification()</code>
Servicio Users	▪ <code>signUp()</code> ▪ <code>updateUserProfile()</code> ▪ <code>getUserProfile()</code> ▪ <code>createAlert()</code> ▪ <code>dropAlert()</code> ▪ <code>findAlertsByUser()</code> ▪ <code>getAlert()</code> ▪ <code>getUsersToAlert()</code>	-
Servicio Courses	▪ <code>createCourse()</code> ▪ <code>modifyCourseDetails</code> ▪ <code>openEnrollment()</code> ▪ <code>closeEnrollment()</code> ▪ <code>enrollInCourse()</code> ▪ <code>closeGradeReports()</code> ▪ <code>closeCourse()</code> ▪ <code>findCourses()</code> ▪ <code>getCourseById()</code> ▪ <code>getEnrolledStudents()</code> ▪ <code>getEnrollmentByCourse()</code>	

Servicio Microcredentials	▪ requestCourseMicrocredential s() ▪ approvePendingMicrocredenti al() ▪ rejectPendingMicrocredential()	



Durante la descomposición, podemos ver la necesidad de añadir operaciones internas que los microservicios deben ofrecer, algo que no habíamos identificado inicialmente.

Algunos ejemplos:

- *getUsersToAlert*: Una operación ofrecida por el *Servicio de Users* para recuperar a los usuarios y notificarlos en caso de que se añada una unidad o se cancele una solicitud de alquiler de un producto con una alerta activa.

- *getRentReservations*: Una operación ofrecida por el *Servicio de ProductCatalog* para recuperar reservas activas de una unidad producto.
- *hasActiveReservations*: Una operación ofrecida por el *Servicio de ProductCatalog* para determinar si un producto tiene unidades con reservas activas.
- *getEnrollmentDetails*: una operación ofrecida por el Servicio de Courses para recuperar el detalle de un curso superado por un alumno y su calificación, para generar los datos de la microcredencial.

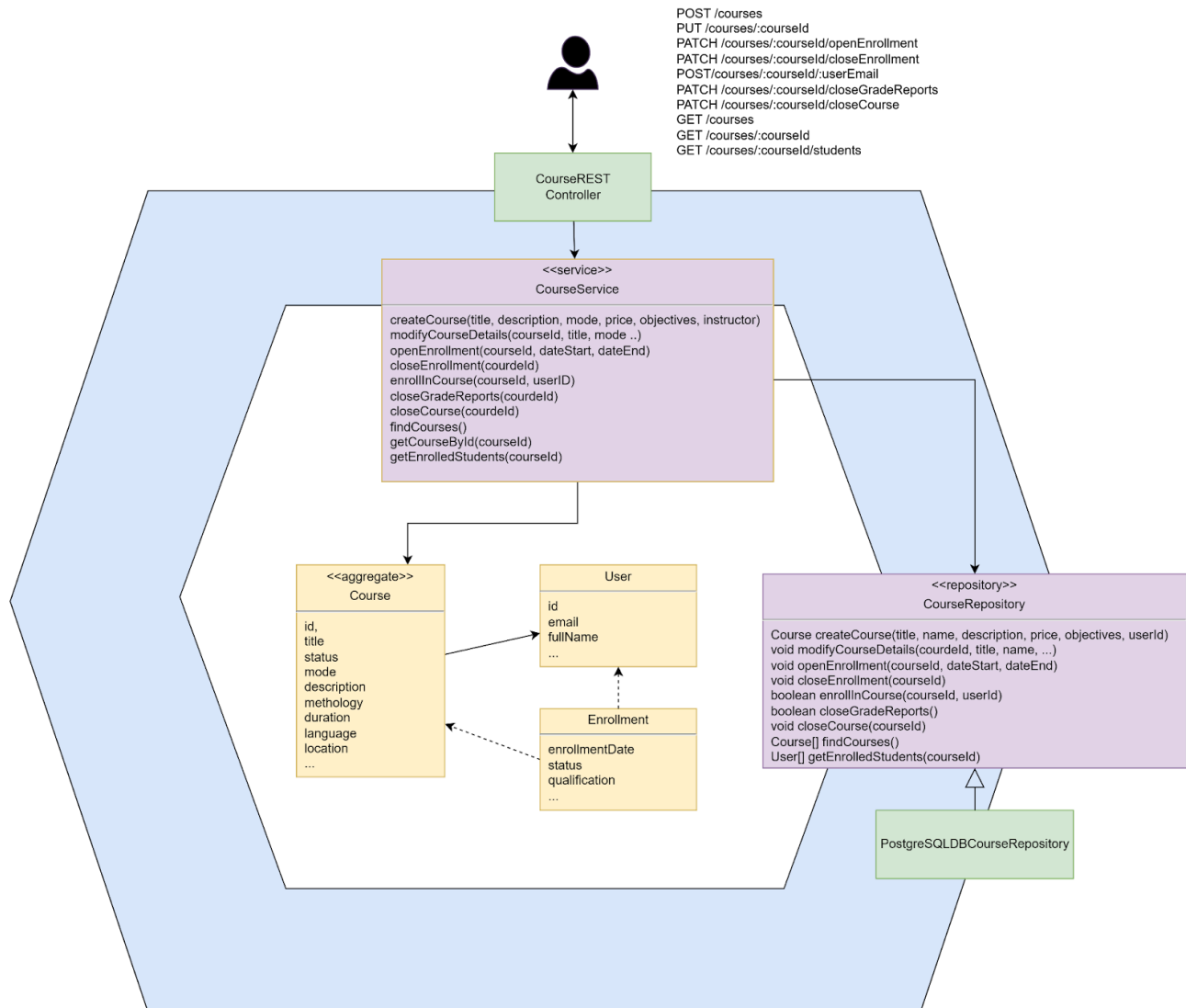
El diagrama muestra adaptadores de ejemplo reales que podrían ser utilizados por los servicios de pago y notificación.

Ejercicio 3 (2.5 puntos)

Basándonos en la descomposición realizada en el ejercicio anterior, detalla el **diseño** del servicio de cursos, teniendo en cuenta:

- El diseño debe realizarse utilizando una **arquitectura hexagonal**.
- El diagrama debe representar los **protocolos de comunicación** con el servicio (REST, etc.) y los **métodos y atributos** de las diferentes clases del servicio (no es necesario indicar los tipos).

Solución



Consideraciones:

- En todas las operaciones, los parámetros se transmiten mediante parámetros de tipo path.
- En el payload (petición), se envían datos del modelo de curso, por ejemplo, la fecha de inicio y la fecha de fin en las operaciones openEnrollment/closeEnrollment (PATCH), o el modelo completo del curso en la petición POST/**PUT**/courses.
- Todas las operaciones que modifiquen el estado del curso (como abrir o cerrar el periodo de inscripción, o cerrar el curso) se gestionan mediante una única operación PATCH. También se considera correcto mapear estas como dos operaciones específicas POST o PUT.

Fuentes para resolver el ejercicio:

- Libro "Microservices Patterns": Capítulo 3, sección "Especificación de APIs REST".
- Libro "Microservices Patterns": Capítulo 5, sección "Diseñando un modelo de dominio usando los patrones agregados DDD".

Ejercicio 4 (1.5 puntos)

¿Qué patrón o patrones aplicarías para gestionar el cierre del curso, la finalización de informes de calificaciones y la generación de microcredenciales, tal y como se describe en la documentación del proyecto?

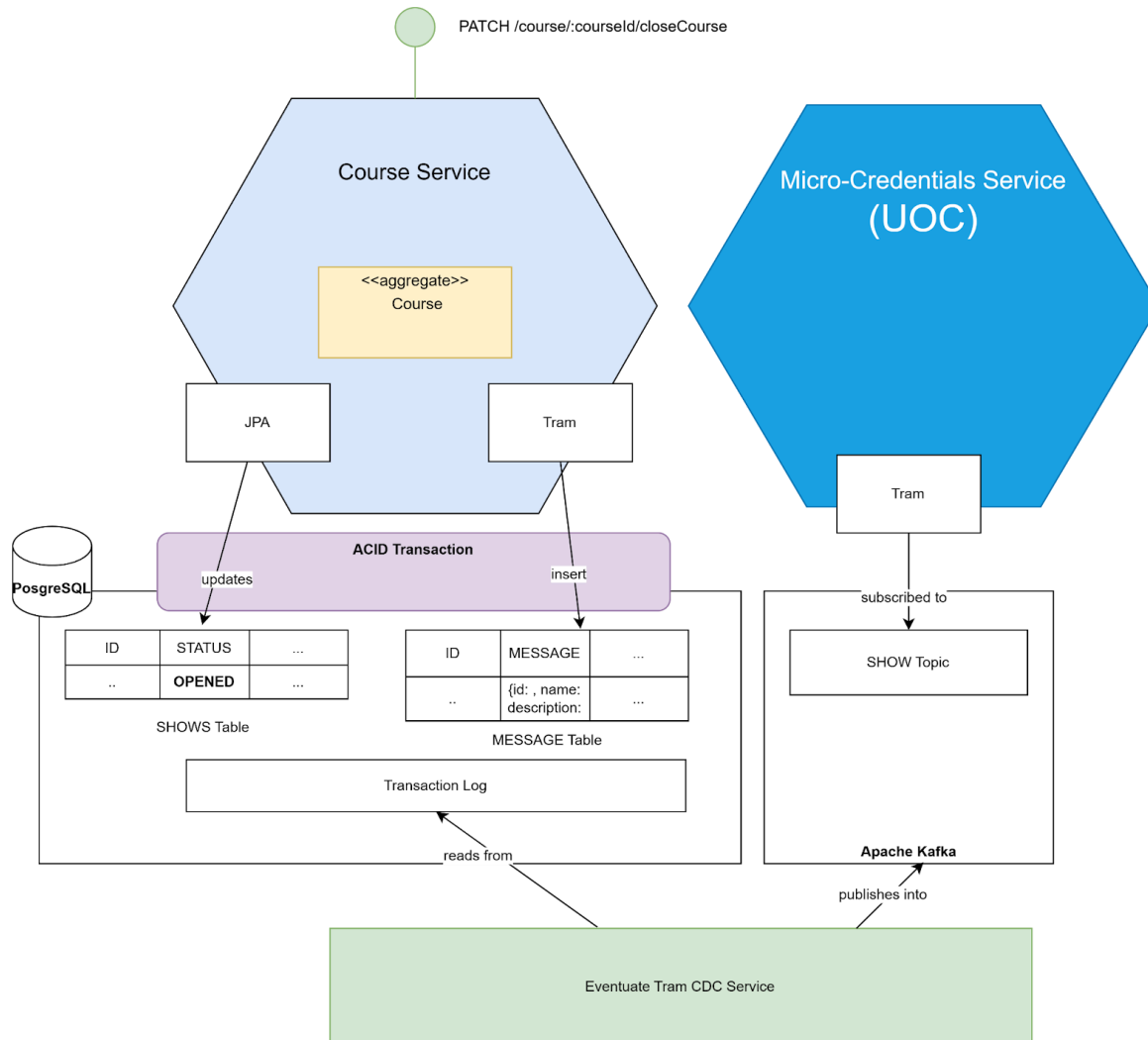
Representa en un diagrama la solución de este enfoque, utilizando el patrón o patrones de microservicios más apropiados. Justifica tus decisiones.

Solución

El patrón elegido para abordar el problema es la **Transactional Outbox**:

<https://microservices.io/patterns/data/transactional-outbox.html>

Cuando uno o varios registros de estudiantes se procesan para un curso, cada registro debe ser validado con éxito antes de que el curso pueda cerrarse. Si algún registro del conjunto encuentra un problema, todos los registros previamente procesados de ese curso deben revertirse hasta que se resuelva el problema.



Este comportamiento puede implementarse usando el **patrón Transactional Outbox**, que asegura atomicidad publicando un evento relacionado con el problema de validación de registros en una tabla de mensajes OUTBOX.

Se puede usar un **patrón Polling Publisher** para procesar y volver a publicar el evento una vez que todos los registros hayan sido validados con éxito, permitiendo que el cierre completo del curso continúe y la generación de microcredenciales.

Fuentes:

- Libro "Microservicios Patrones": Capítulo 3, sección "Comunicación mediante mensajería asíncrona".

- Libro "Microservicios Patrones": Capítulo 5.3, sección "Eventos de dominio editorial".

Ejercicio 5 (2 puntos)

El exitoso lanzamiento del proyecto **Photo&Film4You** ha despertado el interés de **Koursera**, una empresa consolidada conocida por ofrecer cursos online sobre una amplia variedad de temas. Koursera está explorando la creación de nuevos cursos centrados en cine y fotografía, donde los estudiantes tendrían la opción de alquilar equipos profesionales directamente a Photo&Film4You para seguir las lecciones prácticas.

Koursera quiere integrar su plataforma con la de **Photo&Film4You** para ofrecer a sus estudiantes la posibilidad de reservar y gestionar alquileres de equipos directamente desde su plataforma. Si se llega a un acuerdo, esto representaría una oportunidad para llegar a un público más amplio dentro de la industria cinematográfica independiente, a cambio de una pequeña comisión por cada transacción generada a través de **Koursera**.

La dinámica operativa implicaría que, al gestionar una grabación dentro de **Koursera**, los usuarios podrían acceder a una sección de alquiler de equipos. A partir de ahí, **Koursera** consultaría el catálogo de **Photo&Film4You** en tiempo real, junto con otros proveedores, y mostraría los productos disponibles. Si un usuario muestra interés en cualquiera de los modelos de alquiler disponibles en **Photo&Film4You**, la reserva se realizará directamente desde **Koursera**, y el procesamiento de pagos y las actualizaciones de inventario se gestionarán automáticamente por nuestra plataforma.

Aunque la dirección ve esta propuesta con buenos ojos, el Director de Tecnología (CTO) tiene algunas reservas. Durante una reunión técnica inicial con el **CTO de Koursera**, se observó que su solicitud se basa en servicios que requerirían consultar el catálogo de productos **Photo&Film4You**, reservas y gestión de alquileres. Sin embargo, durante las primeras pruebas de integración se detectaron problemas críticos: los alquileres cancelados no siempre liberan stock, los pagos confirmados se realizan sin disponibilidad real y las notificaciones de disponibilidad no siempre llegan a los usuarios que solicitaron alertas. Estos problemas generan inconsistencias en la plataforma, afectando la fiabilidad del servicio y la experiencia del usuario.

A partir de este escenario, nuestra tarea es identificar y justificar un patrón adecuado para gestionar la gestión distribuida de transacciones dentro de nuestra plataforma, considerando las restricciones acordadas, y analizar sus beneficios y desventajas.

Entrega

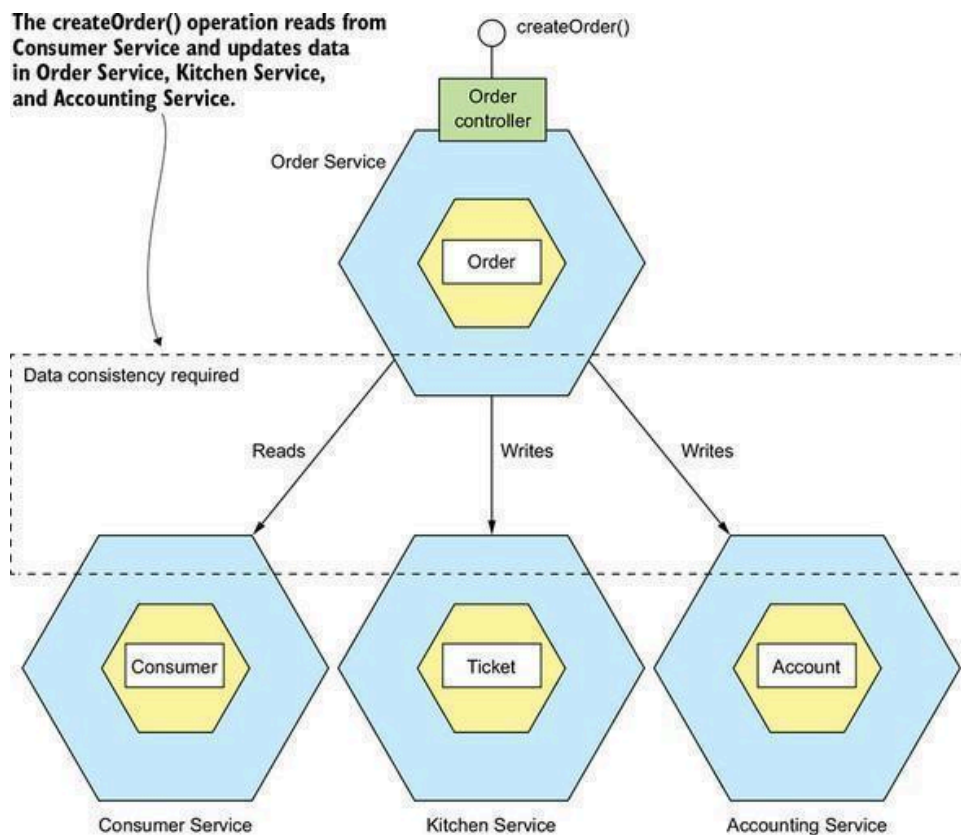
La respuesta a este ejercicio debe enviarse en formato de texto PDF.

Solución

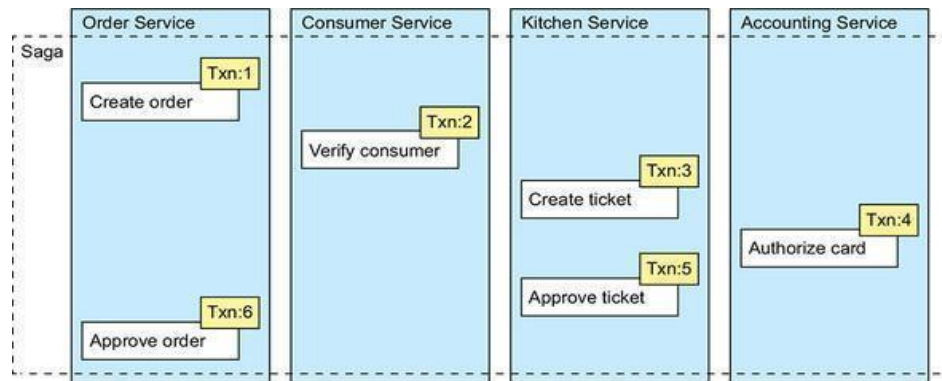
El patrón elegido para abordar el problema dado es **SAGA**:

<http://microservices.io/patterns/data/saga.html>

El patrón SAGA ayuda a mantener la coherencia entre servicios y datos. Es una secuencia de transacciones coordinadas que ayuda a actualizar de forma fiable los datos distribuidos entre múltiples servicios. Al procesar una solicitud, el servicio no interactúa de forma sincrónica con otros servicios, sino que envía mensajes asíncronos a otros servicios, asegurando un acoplamiento flexible entre ellos.

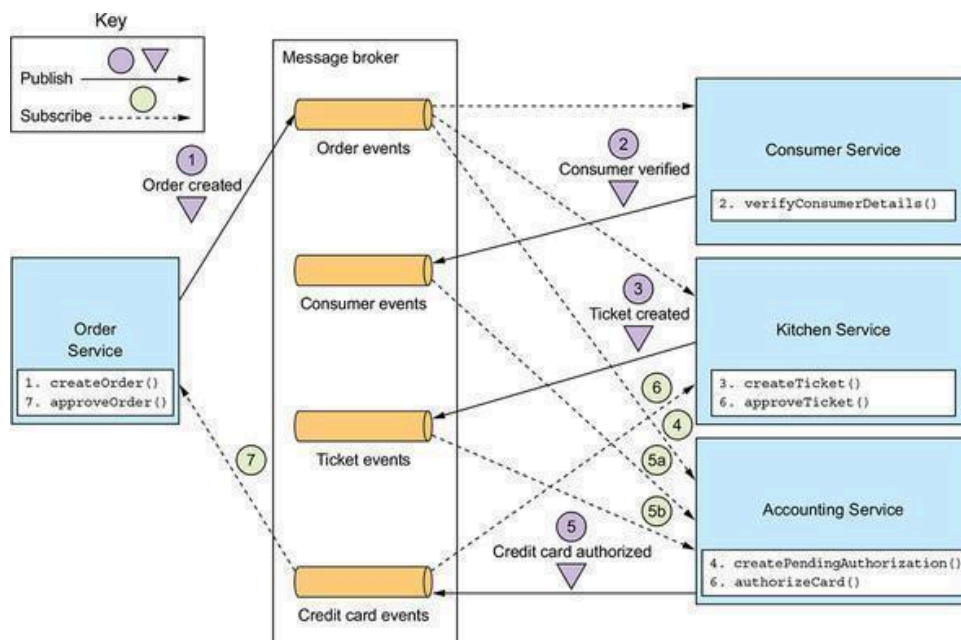


Dado que cada servicio tiene su propia base de datos, debemos garantizar la coherencia de los datos entre estas bases de datos. Una vez iniciada una operación del sistema, la finalización de una transacción desencadena la ejecución de la siguiente.



Hay dos formas de estructurar la lógica de coordinación: **coreografía** u **orquestración**. Cada opción tiene sus pros y sus contras, y ambas serán aceptadas siempre que estén debidamente justificadas. En esta solución, presentamos la justificación para elegir el patrón SAGA con **lógica de coreografía**, que se recomienda para SAGA más simples (en cualquier otro caso, se recomienda orquestración).

La coreografía distribuye la toma de decisiones y la secuenciación entre todos los participantes en la saga, con la comunicación principalmente a través del intercambio de eventos.



El flujo sería el siguiente:

- Se crea una orden y se publica un evento inicial con un estado de aprobación pendiente.

- Los datos de los consumidores se verifican y se publica un evento.
- El pedido se valida, se crea un ticket en estado pendiente y se publica un evento.
- Se crea una autorización de pago y, tras validar el pedido, se procesa el pago y se publica un evento.
- Finalmente, si el pago está autorizado, el estado del pedido y los tickets asociados se actualizan a aceptado/aprobado.

La mensajería asíncrona garantiza que todos los pasos de una saga se ejecuten, incluso si uno o más servicios están temporalmente inaccesibles.

Sin embargo, también existen otras ventajas:

- **Simplicidad:** Los servicios publican eventos al crear, actualizar o eliminar datos, sin necesidad de un orquestador central.
- **Bajo acoplamiento:** Los servicios se suscriben a eventos y reaccionan a ellos sin depender directamente unos de otros.

Las principales desventajas son:

- **Complejidad:** La lógica está distribuida entre los servicios, lo que dificulta su comprensión y depuración.
- **Dependencias cíclicas:** Los servicios responden a eventos entre sí, complicando la arquitectura.

Por otro lado, los posibles inconvenientes de este patrón incluyen la falta de aislamiento entre sagas, ya que otras sagas pueden cambiar datos mientras otra saga se ejecuta o leer datos antes de que se completen las actualizaciones, y la complejidad de revertir cambios cuando ocurre un error, ya que la aplicación debe deshacer los cambios realizados en pasos anteriores (transacciones compensatorias).

Para abordar posibles anomalías, normalmente se utilizan contramedidas y estrategias de diseño para evitarlas. Entre ellas están: actualizaciones perdidas, lecturas sucias (dirty reads) o lecturas borrosas/no repetibles (Fuzzy/nonrepeatable reads).

Existen varios patrones que ayudan a mitigar estas anomalías, como el patrón Circuit Breaker, que evita la propagación de errores deteniendo temporalmente las solicitudes a servicios fallidos, la idempotencia en las operaciones para asegurar que las operaciones puedan repetirse sin causar efectos secundarios no deseados, y la adición de telemetría y observabilidad, que facilitan la detección temprana de errores y problemas de rendimiento al monitorizar el sistema en tiempo real.

Fuentes:

- Patrones de microservicios (Richardson). Capítulo 4: "Gestionar transacciones con sagas."

- Patrón: Saga: <https://microservices.io/patterns/data/saga.html>

Recursos

Recursos Básicos

- Libro "Microservicios Patrones" (Richardson):
 - Capítulo 1. "Escapando del infierno monolítico".
 - Capítulo 2: "Estrategias de descomposición".
 - Capítulo 3: "Comunicación entre procesos en una arquitectura de microservicios".
 - Capítulo 5: "Diseñando lógica de negocio en una arquitectura de microservicios".
 - Capítulo 7 "Implementando consultas en una arquitectura de microservicios".

Recursos Complementarios

- Sitio web: "Un lenguaje de patrones para microservicios".
<https://microservices.io/patterns/index.html>

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Se debe entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PRAC1.pdf*.

Este documento se entregará en el espacio “Entrega de la PRAC1” del aula antes de las **23:59 horas del día 06 de abril de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación

	Excelente (10)	Destacados (7)	Suficiente (5)	Insuficiente (2,5)	Sin respuesta / Todo mal (0)
Ej. 1.1	Enumerar correctamente más de 20 operaciones del sistema	Enumera correctamente entre 10 y 20 operaciones del sistema	Enumerar correctamente menos de 10 operaciones del sistema	Otros (operaciones incorrectas, formato incorrecto, etc....)	Sin respuesta / Todo mal
Ej. 1.2	Especifica correctamente más de 15 comandos	Especifica correctamente entre 5 y 15 comandos	Especificar correctamente menos de 5 comandos	Otros (comandos incorrectos, formato incorrecto, etc.)	Sin respuesta / Todo mal
Ej. 1.3	Modelo de dominio correcto con la mayoría de las entidades y relaciones	Modelo de dominio incompleto, con pocas entidades y/o relaciones pero lo especificado es correcto	Modelo de dominio ligeramente incorrecto (sin atributos, con operaciones, etc.)	Modelo de dominio gravemente incorrecto (entidades del dominio de la solución y no del problema, sin relaciones, etc.)	Sin respuesta / Todo mal
Ej. 2.1	Identificación correcta y diagrama bien elaborado	Identificación no del todo correcta (demasiados o muy pocos subdominios, más de un servicio por subdominio, etc...) y diagrama bien hecho	Identificación incorrecta o diagrama mal elaborado	Identificación incorrecta y diagrama mal elaborado	Sin respuesta / Todo mal

Ej. 2.2	Mapeado correcto de más de 20 operaciones con colaboraciones	Mapeado correcto de más de 20 operaciones sin colaboraciones o con colaboraciones incorrectas	Algunas operaciones son correctas y otras colaboraciones bien hechas	La mayoría de las operaciones son incorrectas y la mayoría de las colaboraciones están mal hechas	Sin respuesta / Todo mal
Ej. 3	Incluye diagrama de servicios del catálogo de productos con arquitectura hexagonal + Describe correctamente los protocolos de comunicación (verbos HTTP) + Incorpora correctamente los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los diferentes elementos + Cada uno de los elementos mencionados ha informado sus atributos y operaciones	Incluye diagrama de servicios del catálogo de productos con arquitectura hexagonal pero no es completamente correcto + Error en algunos protocolos de comunicación (verbos HTTP) + Incorpora con algunos errores los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los diferentes elementos de forma incompleta + Algunos de los elementos mencionados han influido en sus atributos y operaciones	Incluye diagrama de servicios del catálogo de productos con arquitectura hexagonal pero, en gran parte, incorrecto + Error en la mayoría de los protocolos de comunicación (verbos HTTP) + Incorpora muchos errores en los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los diferentes elementos con muchas limitaciones + No se muestran los atributos y operaciones sobre los elementos	Otros: sin diagrama, diagrama 100% incorrecto, arquitectura no hexagonal, etc...+ Sin protocolos de comunicación (verbos HTTP) o totalmente incorrectos + No incorporar los elementos/clases mencionados + No se muestran las relaciones entre los diferentes elementos + No se muestran atributos y operaciones sobre los elementos	Sin respuesta / Todo mal

Ej. 4	Elección correcta, justificación y diagrama	Elección y justificación correctas, pero sin diagrama o con diagrama incorrecto o elección incorrecta con justificación y diagrama correctos	Elección incorrecta con diagrama correcto, pero sin justificación + Elección incorrecta con justificación correcta pero sin diagrama	Elección incorrecta con diagrama incorrecto + Elección incorrecta con justificación incorrecta	Sin respuesta / Todo mal
Ej.5	Entendiendo completamente el patrón. Justifica correctamente el enfoque de coordinación elegido.	Buen entendimiento. Se explica la elección del enfoque de coordinación, pero con menos detalles.	Comprensión básica. Justificación limitada o ausente para el enfoque de coordinación. El flujo es superficial o incompleto.	Comprensión mínima. El flujo está mal descrito. No hay justificación adecuada.	Sin respuesta / Todo mal